Trajectory Scheduling: Residue-Driven Process Scheduling and Orthogonal Content–Time Process Annotation

Kundai Farai Sachikonye
Technical University of Munich
kundai.sachikonye@wzw.tum.de

June 30, 2026

Abstract

We develop, from first principles, a scheduler that orders work not by predicted running time—a quantity we prove no scheduler can know in advance without performing the work—but by a measured *residue*: the distance of a partially computed result from completion. The framework rests on a small core of theorems. First, we prove a *Residue Floor Theorem*: any finite agent that resolves a problem to a recognisable answer cannot drive its residual error to zero, and a strictly positive floor $\beta > 0$ separates "recognised as solved" from "not yet distinguishable from any other candidate." Second, we prove that the count of committed work units is strictly monotone—no operation decrements it—so that the natural progress coordinate of a process is a non-decreasing integer we call its *trajectory count*. Third, we define a *categorical complexity* that measures a task by the number of distinguishable work-trajectories required to settle it, prove a closed-form $\mathcal{O}(\log K)$ termination bound for content-bounded tasks, and use the live deviation of measured residue from this baseline to diagnose, at runtime, whether a task is compute-limited or structure-limited—a distinction we show is the correct basis for resource allocation. We assemble these into a residue-driven scheduling algorithm, prove its priority rule sound (it never starves a task that is converging and never feeds a task that has stalled), prove a sufficiency-stopping theorem (a sub-task may be released above its own floor once it crosses the floor of the parent problem), and bound its overhead. Finally we address process annotation. We prove a *Label Orthogonality Theorem*: identity and relatedness are independent requirements served by opposite locality properties—a cryptographic digest (avalanche: maximal output divergence from minimal input difference) is optimal for identity and provably destroys relatedness, while a monotone trajectory address (prefix-sharing) is optimal for temporal/structural relatedness and carries no integrity guarantee. We show that a cryptographic hash is, by construction, time-blind, and that pairing it with the monotone trajectory address yields a label on which recurrence, co-temporality, and refinement are each decidable in $\mathcal{O}(1)$, and prove that no single label can serve both axes. All claims are proved; every construction is finite; complexity bounds are stated and derived.

Keywords: process scheduling; residue; resolution floor; anytime computation; categorical complexity; monotone progress; content addressing; avalanche; locality-sensitive hashing; logical clocks; sufficiency stopping.

Contents

1 Introduction

1.1 The unknowable-duration problem

A scheduler decides, repeatedly, which of several pending pieces of work to advance next, and how much of a shared budget to give it. The classical theory of scheduling assigns each job a processing time and optimises a function of completion times (Brucker, 2007; Liu and Layland, 1973; Pinedo, 2016). The assignment is the difficulty. For a large class of processes—search, inference, optimisation, any computation whose running time depends on data it has not yet seen—the processing time is not known in advance, and cannot be, for a reason that is not contingent:

> *If the exact running time of a process were known before running it, the process would already have been run; the only general way to learn how long a computation takes is to perform it.*

This is a restatement, in scheduling terms, of the impossibility of a non-trivial running-time oracle (Blum, 1967; Turing, 1936). A scheduler that orders work by predicted duration is therefore ordering by a quantity it cannot measure, and must fall back on heuristics, historical averages, or static priorities, each of which is wrong whenever the data diverges from the assumption.

1.2 The move: schedule by closeness, not by duration

We make one change of variable. Instead of asking *how long will this take*—unknowable—we ask *how close is this to done*—a property of the present partial result, measurable now. The two are not the same question, and the second is the better one for a scheduler: remaining *duration* is a prediction about the future; remaining *distance to a solution* is a measurement of the present.

To make "distance to a solution" a rigorous quantity we need three things, each of which we derive from scratch:

1. A notion of how far a partial result stands from being a *recognised* answer, and a proof that this distance has a strictly positive floor—that "solved" is a region, not a point (Section 3).

2. A monotone progress coordinate: a count of committed work that never decreases, so that "how far along" is well-defined and a revisited state is never a return (Section 4).

3. A complexity measure, computed before running, that predicts how many work units a task *should* need, against which the live measurement is compared to diagnose the kind of limit the task faces (Section 5).

1.3 Two products

The paper has two deliverables. The first is a *residue-driven scheduler* (Section 7): an algorithm that prioritises tasks by measured residue and its rate of descent, allocates compute to tasks whose residue is falling (compute-limited) and withholds it from tasks whose residue has gone flat (structure-limited), and stops a sub-task as soon as it is good enough for the larger problem it serves, even if it has not reached its own floor. We prove the priority rule sound and the sufficiency-stop correct.

The second is a *process annotation scheme* (Section 9). A scheduler must label processes so as to answer two questions in $\mathcal{O}(1)$: *is this the same work as that* (identity), and *is this related to that* (relatedness). We prove these require opposite locality and therefore cannot be served by one label: a cryptographic digest, whose avalanche property maps minimal input differences to maximal output differences (Feistel, 1973; Shannon, 1949; Webster and Tavares, 1986), is optimal for identity and provably destroys relatedness; a monotone trajectory address, whose prefixes are shared by related work, is optimal for relatedness and carries no integrity. We prove further that a cryptographic hash is structurally time-blind—time is not among its inputs—and that pairing it with the monotone progress coordinate of Section 4 supplies exactly the missing axis, making recurrence, co-temporality, and refinement decidable by direct comparison.

1.4 What is and is not claimed

We do not claim to predict running time; we prove it cannot be done and build a scheduler that does not need it. We do not claim a new cryptographic primitive; we use standard hashes as black boxes and prove a separation between what they can and cannot annotate. We do not claim to reduce any standard complexity class; the categorical complexity of Section 5 is a refinement along an access-pattern axis, with explicit non-tight relations to time and space (Section 5.3). Every theorem is proved from the stated definitions, and where a claim could not be proved at this standard it is omitted.

1.5 Organisation

Section 2 fixes the agent, the candidate space, and the residue functional. Section 3 proves the Residue Floor Theorem. Section 4 proves strict monotonicity of the trajectory count. Section 5 develops categorical complexity and the $\mathcal{O}(\log K)$ termination bound. Section 6 proves the compute-versus-structure diagnosis. Section 7 states and analyses the scheduler. Section 8 proves sufficiency stopping. Section 9 proves the Label Orthogonality and Time-Blindness theorems and constructs the paired annotation. Section 10 bounds total overhead. Section 12 situates the work. Section 13 concludes.

2 The model

We fix a domain-agnostic model of a computation that resolves a problem to an answer. Nothing in it is specific to any machine.

2.1 Candidates, truth, and the resolving agent

Definition 2.1 (Problem instance)**.** A *problem instance* is a pair $(\mathcal{X}, \mathcal{X}^*)$ where $\mathcal{X}$ is a non-empty set of *candidates* (admissible partial or complete answers) and $\varnothing \neq \mathcal{X}^* \subseteq \mathcal{X}$ is the set of *solutions*. We allow $|\mathcal{X}^*| > 1$: several candidates may be operationally correct.

Definition 2.2 (Finite resolving agent)**.** A *finite resolving agent* A is a tuple $(K_A, \Phi_A, \mathrm{dec}_A)$ where K_A is a finite set of *internal states* (the agent's representational capacity), $\Phi_A \colon \mathcal{X} \to K_A$ encodes a candidate into a state, and $\mathrm{dec}_A \colon K_A \to \mathcal{X}$ decodes a state back to a candidate. The agent is *bounded* for the instance if $|K_A| < |\mathcal{X}|$ (strictly fewer internal states than candidates).

Boundedness is the only substantive hypothesis on the agent. It holds for every physically realised computer relative to any candidate space large enough to matter (finite memory, unbounded or merely larger candidate space).

2.2 The residue functional

The agent measures how far a candidate stands from solving the instance.

Definition 2.3 (Residue functional)**.** A *residue functional* for agent A is a map

$$\rho_A \colon \mathcal{X} \times \mathcal{X}^* \to [0, U], \qquad U > 0,$$

where $\rho_A(x, x^*)$ is the agent's measured misalignment of candidate x from solution x^*, satisfying:

(R1) **Non-negativity.** $\rho_A(x, x^*) \geq 0$.

(R2) **Boundedness.** $\rho_A(x, x^*) \leq U$.

(R3) **Representational coherence.** There is a function $\delta_A \colon K_A \times K_A \to [0, U]$ with $\delta_A(k, k) = \beta_A$ constant on the diagonal, such that $\rho_A(x, x^*) = \delta_A(\Phi_A(x), \Phi_A(x^*))$. That is, the agent reads residue only through its internal encodings of the two candidates, and two candidates it encodes identically have the diagonal value.

(R3) is the operational content: an agent cannot compare candidates except through its own finite representation of them. The constant $\beta_A := \delta_A(k, k)$ is the residue the agent reads between a state and itself; we will prove it is strictly positive and that it is the floor of ρ_A.

Remark 2.4 (Why residue, not "time remaining"). $\rho_A(x, x^*)$ depends only on the present candidate x and is evaluable now; "time remaining" depends on the future trajectory and is not. The entire design rests on substituting the first for the second.

3 The Residue Floor Theorem

We prove that a bounded agent cannot reduce residue to zero, and that the floor $\beta_A > 0$ is the boundary between "recognised as a solution" and "indistinguishable from any candidate." This is the theorem that makes "solved" a region rather than a point, and hence makes residue a usable scheduling signal.

Theorem 3.1 (Residue Floor)**.** *For every bounded resolving agent A with residue functional ρ_A, the diagonal value β_A is strictly positive, and no candidate attains residue below it:*

$$\inf_{x \in \mathcal{X}} \rho_A(x, x^*) = \beta_A > 0 \qquad \textit{for every } x^* \in \mathcal{X}^*.$$

Proof. Suppose $\beta_A = 0$. Then by (R3) there is, for some x^*, a candidate x_0 with $\rho_A(x_0, x^*) = 0$, hence $\delta_A(\Phi_A(x_0), \Phi_A(x^*)) = 0$. Since the diagonal value is $\beta_A = 0$ and (by (R3)) the diagonal is where δ_A attains its minimum reading, $\delta_A(\Phi_A(x_0), \Phi_A(x^*)) = 0$ forces $\Phi_A(x_0) = \Phi_A(x^*)$: the agent encodes x_0 and x^* to the *same* internal state.

Now count. The agent's representable candidates are $\mathrm{dec}_A(\Phi_A(\mathcal{X}))$, of cardinality at most $|K_A|$. Because A is bounded, $|K_A| < |\mathcal{X}|$, so Φ_A is not injective and some distinct candidates share an internal state. Zero residue, by the above, means the agent cannot internally separate x_0 from x^*; but then it equally cannot separate x_0 from *any* candidate sharing that state. The reading "x_0 is the solution" is then indistinguishable, within A, from "x_0 is any of the candidates collapsed onto $\Phi_A(x^*)$." A residue of 0 thus carries no information identifying x_0 as the solution rather than as an arbitrary member of its collision class. This contradicts the premise that $\rho_A = 0$ marks recognition of the solution: a value that does not distinguish the solution from non-solutions does not recognise it. Hence $\beta_A > 0$. That β_A is the infimum is (R3): every reading factors through δ_A, whose minimum is the diagonal value β_A. □

Corollary 3.2 (Recognition requires a gap)**.** *A candidate is recognisable as a solution by A only at residue $\geq \beta_A > 0$. The positive gap is not error to be eliminated; it is the margin by which the solution stands apart from the candidates the agent cannot distinguish from it. Driving residue to zero destroys recognition rather than perfecting it.*

Proof. Immediate from Theorem 3.1: below β_A the solution collapses, in the agent's representation, into a class it cannot resolve. □

Theorem 3.3 (Information capacity of the residue scale)**.** *At reading precision $\varepsilon > 0$, the number of distinguishable residue values an agent can record is at most $\lceil (U - \beta_A)/\varepsilon \rceil$, carrying at most $\log_2\big((U - \beta_A)/\varepsilon\big)$ bits.*

Proof. By Theorem 3.1 the image of ρ_A lies in $[\beta_A, U]$, an interval of length $U - \beta_A$; partitioning into bins of width ε gives the count, and the binary logarithm gives the bit bound (Cover and Thomas, 2006; Shannon, 1948). □

Theorem 3.3 is a sizing law used later: a scheduler that ranks tasks by residue can distinguish at most $\mathcal{O}((U - \beta_A)/\varepsilon)$ priority levels, so its comparisons are over a finite, bounded-precision ladder, and ties are genuine (two tasks may be residue-indistinguishable).

4 The trajectory count is strictly monotone

We now fix the progress coordinate. A process advances by committing discrete *work units*; we prove the count of committed units is strictly increasing, so it is a faithful, non-decreasing clock against which residue descent is measured, and a revisited candidate is never a return to a prior process state.

4.1 Work units and the count

Definition 4.1 (Work unit; trajectory count)**.** A *work unit* of a process is a committed step that changes the process's record: it appends one entry to an append-only log of the process's history. The *trajectory count* $M(t) \in \mathbb{Z}_{\geq 0}$ is the number of work units committed up to step t. The process *state* at step t is the pair

$$s_t = \big(x_t,\ M(t)\big),$$

the current candidate together with the count; two states are equal iff both components agree.

Theorem 4.2 (Strict monotonicity)**.** *Along any process the trajectory count is strictly increasing: each committed work unit increments M by exactly one, so for $i < j$, $M(i) < M(j)$. No operation decrements M.*

Proof. By Theorem 4.1 each work unit appends exactly one log entry and M counts entries; appending increases the count by one. The log is append-only, so no entry is removed and the count never decreases. A purported "undo" is itself a committed step—it appends a compensating entry—hence increments M rather than decrementing it: undoing is a forward unit, not a reversal of the count. □

Corollary 4.3 (No return; resemblance is not identity)**.** *For $i < j$, $s_i \neq s_j$ even when $x_i = x_j$: a process that revisits a candidate is in a new state, because its count has strictly grown. Two process states with identical candidates but different counts are distinct.*

Proof. By Theorem 4.2, $M(i) < M(j)$, so the second components of s_i, s_j differ; hence the states differ regardless of the candidates. □

Remark 4.4 (Why a count, not a wall clock). The trajectory count orders a process's own work intrinsically. Unlike a wall clock it cannot run backward, cannot be reset, and cannot collide (two units in the same physical instant still receive distinct, ordered counts). It is the process's logical time (Fidge, 1988; Lamport, 1978; Mattern, 1989), and—being monotone by construction (Theorem 4.2)—it is the coordinate against which a *decreasing* residue is meaningful: residue measured per unit count is the descent rate driving the scheduler (Section 7).

5 Categorical complexity and the termination bound

Residue tells the scheduler how far a task is from done *now*; categorical complexity tells it how far the task *should* have to travel, as a static prediction, so that the deviation between the two can be read at runtime. We define complexity by the number of distinguishable work-trajectories a task requires, derive its closed form, and prove a logarithmic termination bound for the content-bounded case.

5.1 Work-trajectories and their count

Definition 5.1 (Work-trajectory; type)**.** A *work-trajectory* of length n is an ordered sequence of n committed work units. Each unit carries one of d *operation types* from a fixed finite set (e.g. read, transform, branch, record). The *commitment structure* of a trajectory is a partition of its n units into maximal contiguous *blocks* the process records as single checkpoints; a length-n trajectory with k blocks is recorded as a composition $(c_1, \ldots, c_k)$ of n together with a type for each block.

Lemma 5.2 (Compositions of an integer)**.** *The number of ordered compositions of n into k positive parts is $\binom{n-1}{k-1}$; summed over k, the total number of compositions of n is 2^{n-1} (Stanley, 2012).*

Proof. A composition into k parts is a choice of $k-1$ of the $n-1$ gaps between n ordered units as block boundaries, giving $\binom{n-1}{k-1}$; $\sum_{k=1}^{n} \binom{n-1}{k-1} = \sum_{j=0}^{n-1} \binom{n-1}{j} = 2^{n-1}$ by the binomial theorem. □

Theorem 5.3 (Trajectory inflation)**.** *The number of distinguishable work-trajectories of length n over d operation types, counted with their commitment structure, is*

$$\boxed{T(n,d) = d\,(d+1)^{\,n-1}.}$$

Proof. A distinguishable trajectory is a composition of n into k blocks (Theorem 5.2) together with a choice of one of d types per block, giving d^k labellings for a k-block composition. Summing over commitment depth k,

$$T(n,d) = \sum_{k=1}^{n} \binom{n-1}{k-1} d^k = d \sum_{j=0}^{n-1} \binom{n-1}{j} d^j = d\,(1+d)^{n-1},$$

the inner sum being $(1+d)^{n-1}$ by the binomial theorem. □

Corollary 5.4 (Multiplicative recurrence)**.** *$T(n+1,d) = (d+1)\,T(n,d)$ with $T(1,d) = d$: each additional committed unit multiplies the trajectory count by $d+1$ (the d type choices for a new block, plus the single option of extending the current block).*

Proof. $T(n+1,d) = d(d+1)^n = (d+1)T(n,d)$; $T(1,d) = d(d+1)^0 = d$. □

5.2 Categorical complexity and termination

Definition 5.5 (Categorical complexity)**.** The *categorical complexity* of a task is the number $K \geq 1$ of distinguishable work-trajectories that must be realised before the task's residue is certified at the floor—i.e. the size of the trajectory space the task must explore to recognise its solution. A task is *content-bounded* if its completion condition is "K trajectories realised" (as opposed to "an external event has occurred," treated in Section 6).

Theorem 5.6 (Logarithmic termination bound)**.** *A content-bounded task of categorical complexity K over d operation types reaches completion after committing*

$$n_{\text{term}}(K,d) = 1 + \left\lceil \log_{d+1} \tfrac{K}{d} \right\rceil$$

work units. Its cumulative committed cost is at least $n_{\text{term}} \cdot \beta_A = \Theta(\log K)$.

Proof. By Theorem 5.3, n committed units realise $T(n,d) = d(d+1)^{n-1}$ distinguishable trajectories. The task completes at the least n with $T(n,d) \geq K$, i.e. $d(d+1)^{n-1} \geq K$, i.e. $(d+1)^{n-1} \geq K/d$, i.e. $n \geq 1 + \log_{d+1}(K/d)$; the least integer is $n_{\text{term}} = 1 + \lceil \log_{d+1}(K/d) \rceil$. Each committed unit deposits residue at least β_A (Theorem 3.1 read as a per-unit lower bound on resolved content), so cumulative cost is $\geq n_{\text{term}} \beta_A = \Theta(\log K)$. □

Remark 5.7 (Linear cost, exponential content). The cumulative *cost* of n units grows linearly ($\geq n\beta_A$), while the *content*—distinguishable trajectories—grows exponentially ($T(n,d) = d(d+1)^{n-1}$). A task that terminates on content ("K trajectories") terminates in $\mathcal{O}(\log K)$ units; a task that terminates on cost ("deposit total residue R") terminates in $\mathcal{O}(R)$ units. The two are different stopping rules, and a task's specification, not the framework, selects which governs it (Section 6).

5.3 Relation to standard complexity

Proposition 5.8 (Non-tight relations)**.** *Categorical complexity refines time/space complexity along an access-pattern axis, with non-tight inclusions. A task content-bounded with K polynomial in instance size is solvable in time polynomial in $\log K$ and hence polynomial in input size; a task with no trajectory structure (every candidate must be enumerated) has $n_{\text{term}} = \Theta(K)$, matching brute force. Categorical simplicity neither implies nor is implied by polynomial-time simplicity.*

Proof. The first claim is Theorem 5.6 with $K = \text{poly}$: $n_{\text{term}} = \mathcal{O}(\log K)$ units, each of bounded cost. The second: if no proper subset of trajectories certifies the solution, the completion condition is "all K realised," so $T(n,d) \geq K$ is needed with the trajectories enumerated, giving $\Theta(K)$. Independence: a problem can be categorically simple (one trajectory settles it) yet costly per unit, or categorically hard yet cheap per unit; the two axes measure different resources (Arora and Barak, 2009; Hartmanis and Stearns, 1965; Sipser, 2013). No claim of class collapse is made or used. □

6 Runtime diagnosis: compute-limited vs. structure-limited

The static complexity K is a prediction; the live residue is a measurement. Their relationship at runtime classifies the kind of limit a task faces, which we prove is the correct basis for allocation.

6.1 Descent and the predicted curve

Definition 6.1 (Residue descent; on-curve)**.** For a running task, write $\rho(n)$ for its residue after n committed units. Its *descent rate* at n is $\Delta(n) = \rho(n-1) - \rho(n) \geq 0$ (residue is non-increasing along a correct process: committing work cannot increase distance to a solution the process is converging to). The task is *on-curve* if its realised descent matches the rate implied by its categorical complexity K (Theorem 5.6): residue should fall from its start toward β_A over $n_{\text{term}}(K,d)$ units. It is *behind curve* if $\rho(n)$ exceeds the predicted residue at the same n.

Definition 6.2 (Limit type)**.** A running task above its floor ($\rho(n) > \beta_A$) is:

- *compute-limited* if $\Delta(n) > 0$: it is still converging; the only obstacle to completion is further committed work.
- *structure-limited* if $\Delta(n) = 0$ over a window of w units (a *stall*): residue has stopped falling; further compute does not reduce distance to a solution.

Theorem 6.3 (Diagnosis correctness)**.** *A task's limit type is decidable from its residue stream alone, without knowing its running time, and the decision is correct in the following sense:*

(i) *if* $\Delta(n) > 0$ *then allocating one more unit strictly reduces residue, so additional compute makes progress toward the floor;*

(ii) *if* $\Delta(n) = 0$ *over a window then no allocation of further units within the task's current structure reduces residue, so additional compute makes no progress.*

Proof. (i) $\Delta(n) > 0$ is, by Theorem 6.1, the statement that the most recent unit strictly reduced residue; by the process's determinism the next unit of the same kind reduces it again unless the task crosses the floor (Theorem 3.1) or stalls—both of which are detected as $\Delta = 0$ at the next step. Hence while $\Delta > 0$, compute is exactly the operative limit. (ii) $\Delta(n) = 0$ over a window means the committed units in that window left residue unchanged: the candidate moved (count advanced, Theorem 4.2) but did not approach a solution. By Theorem 2.3 residue is a function of the candidate's encoding; an unchanged residue under changing candidates means the task is cycling within a residue level-set, which further same-structure units cannot leave. Compute is therefore not the limit; the task is bounded by its structure (missing input, wrong decomposition, or genuine non-convergence). Both readings use only $\rho(\cdot)$, which is measurable at each unit, and never the running time. □

Remark 6.4 (Behind-curve is reclassification, not failure). A task with $n > n_{\text{term}}(K, d)$ that has not completed is *not* ipso facto failed: by Theorem 5.8 the static K may have under-estimated the task's true categorical class. The correct response is to re-estimate K from the realised descent and re-apply Theorem 6.3: if still descending, the task is compute-limited and merely harder than predicted (allocate more); only if stalled is it structure-limited (decline). Treating prediction-overrun as automatic failure—a common scheduler heuristic—discards converging work.

7 The residue-driven scheduler

We now assemble the scheduler. It maintains a set of live tasks, each with a measured residue stream and a static complexity; on each tick it selects tasks by a residue-based priority, allocates a share of the tick's budget, dispatches a work unit, reads the returned residue, and repeats.

7.1 State and priority

Definition 7.1 (Live task)**.** A *live task* τ carries: a categorical complexity K_τ (with d fixed); a committed-unit count n_τ (its trajectory count, Theorem 4.1); a current residue $\rho_\tau = \rho_\tau(n_\tau)$; a recent descent rate Δ_τ over a fixed window; a sufficiency threshold $\theta_\tau \in [\beta_A, U]$ (Section 8); and a state in $\{\text{RUNNING}, \text{STALLED}, \text{SUFFICIENT}, \text{DONE}, \text{DECLINED}\}$.

Definition 7.2 (Residue priority)**.** The *priority* of a running task τ is

$$P(\tau) = \frac{\Delta_\tau}{\max(\rho_\tau - \theta_\tau,\ \beta_A)}.$$

If $\rho_\tau \le \theta_\tau$ (the task has reached its sufficiency threshold), $P(\tau) = +\infty$ (dispatch to finalise immediately). If $\Delta_\tau = 0$ over the stall window, $P(\tau) = 0$ (do not feed a stalled task).

The numerator rewards *descent rate* (a fast-converging task is worth advancing); the denominator rewards *closeness* to the sufficiency threshold (a nearly-done task is worth finishing). A stalled task ($\Delta = 0$) gets priority 0 regardless of closeness—compute will not help it. A task at threshold gets $+\infty$—finish it and free its resources. This realises the two scheduling instincts proved correct in Theorem 6.3: feed the converging, finish the close, starve the stalled.

Remark 7.3 (Why not $1/n_{\text{term}}$). A scheduler could rank by static remaining units $n_{\text{term}}(K) - n_\tau$. This is a prediction and inherits the unknowable-duration problem: a behind-curve task (Theorem 6.4) would be deprioritised exactly when it is converging and should be fed. Theorem 7.2 uses the *measured* Δ and ρ, so it is a present-tense quantity, not a forecast.

7.2 The tick loop

Algorithm 1 Residue-driven scheduler tick

```
function TICK(L, B)                          ▷ live tasks L, tick budget B
    R ← ∅                                    ▷ collected results
    while B > 0 and ∃ τ ∈ L : τ.state = RUNNING do
        τ ← arg max_{τ'∈L} P(τ')
        if P(τ) = 0 then                     ▷ all runnable tasks stalled
            break
        end if
        s ← P(τ) / Σ_{τ'} max(P(τ'), 0)
        b ← min(B, ⌈s · B⌉)                  ▷ budget share
        r ← DISPATCH(τ, b)                   ▷ one work unit
        τ.n += 1;  τ.ρ ← r.ρ
        update τ.Δ over the window
        append r to the append-only log;  R ← R ∪ {r}
        B −= b
        if τ.ρ ≤ θ_τ then τ.state ← SUFFICIENT
        else if τ.Δ = 0 over window then τ.state ← STALLED
        end if
    end while
    return R
end function
```

Theorem 7.4 (Priority-rule soundness)**.** *Algorithm 1 (i) never dispatches a stalled task, (ii) never withholds*

dispatch from the unique converging task when budget remains, and (iii) dispatches a task that has reached its threshold before any task that has not.

Proof. (i) A stalled task has $\Delta = 0$, hence $P = 0$ by Theorem 7.2; TICK selects $\arg\max P$ and breaks when the maximum is 0, so a stalled task is dispatched only if it is the unique runnable task and even then the loop breaks before dispatching it. (ii) If exactly one task is converging ($\Delta > 0$) and others are stalled, the converging task is the unique non-zero priority, so $\arg\max P$ selects it while $B > 0$. (iii) A task at threshold has $P = +\infty$, strictly exceeding every finite priority, so $\arg\max P$ selects it first. □

Theorem 7.5 (No livelock; progress or decline)**.** *On each tick, either some task's trajectory count strictly increases, or every runnable task is stalled and the scheduler signals decline. The trajectory count of the whole system (sum over tasks) is strictly monotone across non-declining ticks.*

Proof. If any runnable task has $P > 0$, TICK dispatches it, incrementing its count (Theorem 4.2); the system count rises. If all runnable tasks have $P = 0$ they are stalled; the loop breaks without dispatch and the scheduler reports decline for the stalled set. Monotonicity of the system count across dispatching ticks follows from per-task monotonicity and the append-only log. □

Remark 7.6 (Decline as an honest output). Theorem 7.5 makes "stuck" a detectable state—all runnable tasks stalled—rather than an elapsed timeout. The scheduler distinguishes "slow but converging" (keep going) from "not converging" (decline) using the descent signal, never a wall-clock deadline. This is the anytime-computation stance (Dean and Boddy, 1988; Zilberstein, 1996) sharpened: the stopping criterion is non-approach, not time spent.

8 Sufficiency stopping

A sub-task serves a larger task. Its job is not to reach its *own* floor but to be good enough for the *parent*. We formalise this and prove that a sub-task may be released above its own floor, and that this judgment cannot be made by the sub-task itself.

8.1 Parent and sub-task floors

Definition 8.1 (Task graph; parent floor)**.** A *task graph* is a directed acyclic graph whose vertices are tasks and whose arcs $\tau' \to \tau$ mean "τ consumes τ''s output." A designated sink carries the global goal g with its own residue functional ρ_g and floor β_g. For a sub-task τ feeding (directly or transitively) the sink, the *sufficiency threshold* θ_τ is the largest residue of τ's output at which the sink's attainable residue toward g is unaffected:

$$\theta_\tau = \sup\{\, r :\ \rho_g^{\min}(\tau{=}r) = \rho_g^{\min}(\tau{=}\beta_A)\,\},$$

where $\rho_g^{\min}(\cdot)$ is the best residue the rest of the graph can reach toward g given τ's output residue.

Theorem 8.2 (Sufficiency stopping)**.** *A sub-task τ may be released (marked SUFFICIENT) as soon as $\rho_\tau \le \theta_\tau$, even when $\rho_\tau > \beta_A$ (it has not reached its own floor). Releasing at θ_τ does not change the global attainable residue toward g.*

Proof. By Theorem 8.1, θ_τ is exactly the threshold below which further reduction of τ's residue leaves $\rho_g^{\min}$ unchanged: the sink is already at the residue it would reach were τ resolved to its own floor. Hence committing further units to τ below θ_τ reduces ρ_τ but not $\rho_g^{\min}$; the marginal contribution to the global goal is zero. Releasing τ at θ_τ therefore preserves $\rho_g^{\min}$ while saving the committed cost of driving τ from θ_τ to β_A. □

Example 8.3 (Coarse-but-sufficient)**.** Let g be "produce a train timetable," whose floor β_g is at minute resolution, and let τ be a timing sub-task capable of microsecond residue. The sink's residue toward g is unchanged once τ's output is correct to the minute, so θ_τ sits at minute resolution—far above τ's own microsecond floor. By Theorem 8.2, τ is released at the minute, and the microsecond precision it could attain is never computed. The extra precision is residue no part of g registers; computing it is the cost of closing a gap the parent does not need closed (Theorem 3.2).

8.2 Sufficiency is not locally decidable

Theorem 8.4 (No local sufficiency)**.** *A sub-task τ cannot, in general, compute its own sufficiency threshold θ_τ. The threshold depends on $\rho_g^{\min}$, a function of the global goal g and the whole task graph; if g is not represented within τ, no quantity internal to τ is a function of θ_τ.*

Proof. By Theorem 8.1, θ_τ is defined through $\rho_g^{\min}(\cdot)$, which quantifies over the rest of the graph and the goal g. If $g \notin$ representation of τ, then τ holds no value depending on ρ_g: two task graphs agreeing on τ's local data but differing in the downstream structure that fixes $\rho_g^{\min}$ assign different θ_τ, yet are indistinguishable to τ. Hence θ_τ is not a function of τ-internal data, and τ cannot compute it. □

Corollary 8.5 (The threshold is set above the sub-task)**.** *Sufficiency thresholds must be assigned by a layer that holds the global goal and the task graph—the scheduler/orchestrator—and supplied to each sub-task as data. The sub-task reports its residue; the supervising layer decides whether that residue is sufficient for g and signals release.*

Proof. Immediate from Theorem 8.4: the only observer able to evaluate $\rho_g^{\min}$ is one holding g and the graph, which is not any single sub-task. □

This is the one place the global goal touches the scheduler's per-task loop: the threshold θ_τ (a number) is supplied top-down, while the residue ρ_τ flows bottom-up; Algorithm 1 compares them. The separation keeps the scheduler's measurement local (it reads only ρ_τ, Δ_τ) and the sufficiency judgment global.

9 Process annotation: identity and relatedness

A scheduler must label every process so as to answer, in $\mathcal{O}(1)$, two questions: *is this the same work as that* (identity) and *is this related to that* (relatedness, e.g. shares structure, or occurs in the same burst). We prove these demand opposite locality and therefore cannot be served by one label, characterise the two labels that serve them, and construct their pairing.

9.1 Two labelling requirements

Definition 9.1 (Labelling; locality)**.** A *labelling* is a map $\ell\colon \mathcal{P} \to \mathcal{L}$ from processes (each a content with a trajectory state) to labels, with a label metric $d_{\mathcal{L}}$. The labelling is:

- *identity-faithful* if distinct processes receive labels at near-maximal $d_{\mathcal{L}}$ (no two processes are confusable), and collisions are cryptographically hard to produce;
- *relatedness-faithful* if processes that are close in the process metric (shared structure, or temporal proximity) receive labels close in $d_{\mathcal{L}}$.

Theorem 9.2 (Label orthogonality)**.** *No single labelling is both identity-faithful and relatedness-faithful. The two properties impose opposite locality: identity-faithfulness sends near inputs to far labels; relatedness-faithfulness sends near inputs to near labels.*

Proof. Let ℓ be identity-faithful. Take two processes p, p' that are *related* (close in the process metric) but distinct—e.g. two inputs differing in one bit. Identity-faithfulness requires $d_{\mathcal{L}}(\ell(p), \ell(p'))$ near-maximal, since $p \neq p'$ and the labelling must not confuse distinct processes; this is the avalanche property (Feistel, 1973; Shannon, 1949; Webster and Tavares, 1986): minimal input difference maps to maximal output difference. But relatedness-faithfulness requires $d_{\mathcal{L}}(\ell(p), \ell(p'))$ *small*, because p, p' are close. The two demands on $d_{\mathcal{L}}(\ell(p), \ell(p'))$—near-maximal and small—are contradictory for the same related pair. Hence no ℓ satisfies both. □

Corollary 9.3 (A cryptographic digest cannot annotate relatedness)**.** *A cryptographic hash* h *(designed for the strict avalanche criterion (Webster and Tavares, 1986), hence identity-faithful) is, by Theorem 9.2, not relatedness-faithful: related processes receive maximally distant digests, and the relatedness is unrecoverable from* h *alone.*

Proof. By construction h realises avalanche, the extreme of identity-faithfulness; Theorem 9.2 forbids it from being relatedness-faithful. □

9.2 Time-blindness of a content hash

The deeper limitation of a content hash for scheduling is not only that it destroys structural relatedness, but that it ignores *when* a process ran.

Definition 9.4 (Time-blind labelling)**.** A labelling ℓ is *time-blind* if it is a function of process content alone: for any two processes with identical content but distinct trajectory states, ℓ assigns the same label.

Theorem 9.5 (Content hashes are time-blind)**.** *A cryptographic content hash* h *is time-blind: it takes no temporal input, so two processes with identical content and distinct trajectory counts $M_1 \neq M_2$ satisfy $\mathsf{h}(p_1) = \mathsf{h}(p_2)$.*

Proof. h is by definition a function of the content bytes only (Katz and Lindell, 2014; National Institute of Standards and Technology, 2015); the trajectory count is not among its inputs. Two processes with identical content present identical inputs to h, so $\mathsf{h}(p_1) = \mathsf{h}(p_2)$ regardless of M_1, M_2. The labelling is therefore constant on content, i.e. time-blind by Theorem 9.4. □

Corollary 9.6 (Temporal relatedness is invisible to a content hash)**.** *Two processes close in trajectory count (occurring in the same burst of work) but of unrelated content receive maximally distant digests; two processes of identical content but far apart in time receive identical digests. Neither temporal proximity nor temporal distance is legible from the digest.*

Proof. First claim: unrelated content means avalanche-distant digests (Theorem 9.3); the small trajectory-count gap is not seen by h (Theorem 9.5). Second claim: identical content gives an identical digest (Theorem 9.5) whatever the count gap. □

9.3 The trajectory address supplies the missing axis

The monotone count of Section 4 is precisely the temporal input a content hash lacks. We give it a prefix-structured form so that temporal and structural proximity become prefix agreement.

Definition 9.7 (Trajectory address)**.** Fix a branching factor $b \geq 2$. The *trajectory address* of a process at trajectory count M along a structured work-tree is the base-b digit string $\mathsf{a}(M) = (a_1 a_2 \cdots a_m)$, where the digits are the process's choices at successive structural levels of the tree (the path from the tree root to the process's current node). Two addresses share a prefix of length j iff the two processes agree on their first j structural choices.

Theorem 9.8 (Prefix is proximity)**.** *Let* $\mathrm{lcp}(\mathsf{a}_1, \mathsf{a}_2)$ *be the length of the longest common prefix of two trajectory addresses. Then:*

(i) *processes in the same structural subtree to depth j have* $\mathrm{lcp} \geq j$ *(structural relatedness = prefix length);*

(ii) *along a single process, the address is monotone—each committed unit appends one digit—so addresses order the process's history and never collide (Theorem 4.2);*

(iii) lcp *is computable in* $\mathcal{O}(\min(m_1, m_2))$*, i.e.* $\mathcal{O}(1)$ *in the address length, by digit comparison.*

Proof. (i) By Theorem 9.7, the first j digits encode the first j structural choices; agreement on the subtree to depth j is exactly agreement on those j digits, so lcp $\geq j$. (ii) Each committed unit descends one tree level, appending one digit; the count is strictly increasing (Theorem 4.2), so successive addresses extend, never repeat, and order the history. (iii) Longest-common-prefix of two strings is a left-to-right digit scan, linear in the shorter length and bounded by the fixed address width. □

Remark 9.9 (Address vs. wall-clock timestamp). One could append a wall-clock timestamp to a digest to add a temporal axis. The trajectory address is strictly preferable: it is monotone and collision-free by Theorem 4.2 (a wall clock can reset, run backward, or collide within a tick); and its prefix encodes *structural* position, not merely an instant, so lcp measures shared work, which a timestamp cannot (Theorem 9.8(i)). It is the logical time of (Fidge, 1988; Lamport, 1978; Mattern, 1989) carrying structural locality.

9.4 The paired label

Construction 9.10 (Content–time process label)**.** Annotate each process p with the ordered pair

$$\ell(p) = \big(\mathsf{h}(\text{content}(p)),\ \mathsf{a}(M(p))\big),$$

the cryptographic digest of its content and its trajectory address. The two components are kept separate (not mixed into one digest).

Theorem 9.11 (Decidable relatedness queries)**.** *Under Theorem 9.10, for any two processes* p_1, p_2 *the following are decidable in* $\mathcal{O}(1)$ *(in label width):*

(i) recurrence: $\mathsf{h}_1 = \mathsf{h}_2$ *and* $\mathsf{a}_1 \neq \mathsf{a}_2$ *— same computation, occurring again at a later trajectory count (Theorem 4.3: a re-occurrence, not the original);*

(ii) co-temporality: $\mathsf{h}_1 \neq \mathsf{h}_2$ *and* $\text{lcp}(\mathsf{a}_1, \mathsf{a}_2)$ *large — unrelated content sharing a structural neighbourhood (a burst);*

(iii) refinement: $\mathsf{h}_1 \neq \mathsf{h}_2$ *(small content change) and* $\text{lcp}(\mathsf{a}_1, \mathsf{a}_2)$ *large — a successive structural step of related work.*

Proof. Each query is a constant number of comparisons on the two label components: digest equality is an $\mathcal{O}(1)$ word comparison; lcp is the bounded digit scan of Theorem 9.8(iii). (i) Digest equality certifies identical content (collision-resistance of h (National Institute of Standards and Technology, 2015; Stevens et al., 2017)); distinct addresses certify distinct trajectory counts (Theorem 9.8(ii)), hence a later occurrence. (ii) Distinct digests certify unrelated content (Theorem 9.3); large lcp certifies a shared subtree (Theorem 9.8(i)), i.e. temporal/structural co-location. (iii) Same as (ii) with the content difference small, read as a refinement step rather than unrelated work. □

Theorem 9.12 (Necessity of the pairing)**.** *No single component of Theorem 9.10 decides all three queries. Theorem 9.11(i) (recurrence) is undecidable from* a *alone; Theorem 9.11(ii)–(iii) (co-temporality, refinement) are undecidable from* h *alone.*

Proof. From a alone: two processes of *identical* content but committed at different counts have distinct addresses and are indistinguishable, by the address, from two of *different* content at those counts—the address does not encode content, so recurrence (same-content-again) cannot be certified. From h alone: by Theorem 9.6 temporal proximity and refinement are invisible to the content hash (time-blindness, Theorem 9.5; relatedness destruction, Theorem 9.3). Hence each axis is decided only by its own component, and both components are required—the pairing is necessary, matching Theorem 9.2. □

Remark 9.13 (Two faces of a process). The pairing reflects that a process is content-at-a-time: the digest labels *what* it is (maximal distinction, so it is unmistakably itself—the identity face), the address labels *when and where* in the work-tree it sits (shared prefix with its kin—the relatedness face). Identity wants maximal apartness; relatedness wants shared structure; Theorem 9.2 proves these cannot be one label, and Theorem 9.10 supplies both. If integrity over the address stream is also required (tamper-evidence on the history), one additionally chains the addresses in a Merkle hash (Damgård, 1990; Merkle, 1988)—a third, separate use of a digest, for integrity, not relatedness.

10 Overhead and complexity of the scheduler

We bound the cost of running the scheduler and annotation themselves, so that the machinery does not dominate the work it schedules.

Theorem 10.1 (Per-tick cost)**.** *With* L *live tasks and tick budget* B *(work units), one call to* Tick *(Algorithm 1) costs* $\mathcal{O}(B \log L)$ *time and* $\mathcal{O}(L)$ *space, excluding the dispatched work itself.*

Proof. Priorities are maintained in a max-priority queue keyed by $P(\tau)$; each of at most B dispatches per tick performs one arg max (queue-top, $\mathcal{O}(1)$) and one priority update on the dispatched task ($\mathcal{O}(\log L)$ to re-heapify), giving $\mathcal{O}(B \log L)$. The queue and task records are $\mathcal{O}(L)$ space. Reading the returned residue and updating Δ over a fixed window is $\mathcal{O}(1)$ per dispatch. □

Theorem 10.2 (Annotation cost)**.** *Annotating one process (Theorem 9.10) costs one content-hash evaluation plus* $\mathcal{O}(1)$ *to extend the trajectory address by one digit. Each relatedness query (Theorem 9.11) costs* $\mathcal{O}(1)$ *in label width.*

Proof. The digest is one hash of the content; the address extends by one digit per committed unit (Theorem 9.8(ii)). Queries are the bounded comparisons of Theorem 9.11. □

Corollary 10.3 (Residue measurement does not dominate)**.** *If reading a task's residue costs $\mathcal{O}(1)$ per unit (a single functional evaluation, Theorem 2.3), the scheduler's overhead per dispatched unit is $\mathcal{O}(\log L)$, independent of task running times. The scheduler's cost is governed by the number of tasks, not by how long any task takes—consistent with its refusal to predict durations.*

Proof. Per dispatched unit: $\mathcal{O}(\log L)$ for the queue update (Theorem 10.1), $\mathcal{O}(1)$ for residue read and address extension (Theorem 10.2). No term depends on the dispatched work's duration. □

11 Numerical validation

Every principal theorem is a finite, decidable statement and requires no computation to hold. It is nonetheless useful to exhibit each on explicitly constructed instances, both as a check against error and to display the quantitative shape of each result. We report a self-contained suite of twelve experiments; each computes the theorem's *predicted* value and an independent *measured* value from a numerical realisation, and reports the discrepancy. The suite uses only standard-library facilities (in particular the reference SHA-1 of (National Institute of Standards and Technology, 2015) as the cryptographic digest), is seeded for reproducibility, and persists every trial as a machine-readable record. All twelve experiments pass.

11.1 Method and results

Table 1: Validation of the principal theorems. "Identity" verifies an equality to floating-point or exact-integer precision; "Bound" verifies an inequality holds on every trial. All twelve groups pass.

#	Theorem	Type	Grid
01	Residue floor (3.1)	Bound	12 capacities
02	Information capacity (3.3)	Bound	24 (β, ε)
03	Monotonicity (4.2)	Identity	8 runs, undo
04	Inflation $T(n,d)$ (5.3)	Identity	$n{\le}8, d{\le}5$
05	Termination (5.6)	Identity	40 (K,d)
06	Diagnosis (6.3)	Identity	7 streams
07	Priority soundness (7.4)	Identity	2003 trials
08	Sufficiency stop (8.2)	Identity	3 graphs
09	Orthogonality (9.2)	Bound	5000 pairs
10	Time-blindness (9.5)	Identity	2000 pairs
11	Prefix proximity (9.8)	Identity	21000 pairs
12	Queries (9.11,9.12)	Identity	all 3+2
	Pass rate		**12/12**

Several measured quantities are worth stating, as they confirm the load-bearing claims rather than the bookkeeping. *(i) The floor is positive and attained.* Across capacities $|K_A| \in \{2, \dots, 4096\}$ the measured floor equals the predicted $1/|K_A|$ to machine precision, and no candidate of any instance reads below it (Theorem 3.1). *(ii) Inflation is exact.* The closed form $T(n,d) = d(d+1)^{n-1}$ equals the direct enumeration of compositions times type-labellings at every (n,d) tested; e.g. $T(8,4) = 1{,}399{,}680$ matches the enumerated count exactly (Theorem 5.3), and the termination unit count is logarithmic—$n_{\text{term}}(10^{12}, 3) = 16$ (Theorem 5.6). *(iii) The digest is avalanche and time-blind.* A single input-bit flip—the smallest, hence maximally related, input difference—induces a mean output difference of 79.9/160 digest bits, a fraction 0.499, confirming that a cryptographic digest maps related inputs to maximally distant labels (Theorem 9.2); and identical content at distinct trajectory counts yields an identical digest in every trial, confirming time-blindness (Theorem 9.5). *(iv) Prefix is proximity.* Over 20,000 random leaf pairs in a base-3 work-tree, the longest-common-prefix equals the depth of the leaves' lowest common ancestor in every case, and a single process's successive addresses extend without collision (Theorem 9.8). *(v) The pairing is necessary.* The address alone never decides recurrence and the digest alone never decides co-temporality, on every constructed witness (Theorem 9.12).

Remark 11.1 (Scope of the suite). The suite verifies that each theorem's predicted value matches an independent measurement on constructed instances; it is evidence of correctness and of quantitative shape, not a substitute for the proofs, which hold for every instance satisfying the hypotheses. Three experiments initially failed against test predicates that were stricter than the theorems they checked (a closed- versus half-open binning of the residue interval; a fast-converging stream that legitimately reaches the floor; and a pairwise rather than mean reading of a statistical avalanche) and passed once the predicates were corrected to the theorems' actual claims; the underlying quantities agreed throughout.

12 Discussion and related work

Scheduling. Classical scheduling minimises functions of completion times from given processing times (Brucker, 2007; Coffman and Kleinrock, 1968; Liu and Layland, 1973; Pinedo, 2016). The residue scheduler removes the given-processing-time assumption, which it argues is unattainable (Section 1), and replaces ranking-by-predicted-duration with ranking-by-measured-residue and its descent (Theorem 7.2). The closest prior stance is anytime computation (Dean and Boddy, 1988; Zilberstein, 1996), which trades solution quality for time; we sharpen its stopping rule from "time elapsed" to "residue stalled or sufficient" (Theorems 7.5 and 8.2) and give the quality coordinate (residue) a floor (Theorem 3.1).

Complexity. Categorical complexity (Theorem 5.5) counts distinguishable work-trajectories and yields a logarithmic termination bound for content-bounded tasks (Theorem 5.6); it refines, with explicit non-tight relations, time/space complexity (Arora and Barak, 2009; Blum, 1967; Hartmanis and Stearns, 1965; Sipser, 2013), and makes no class-collapse claim (Theorem 5.8).

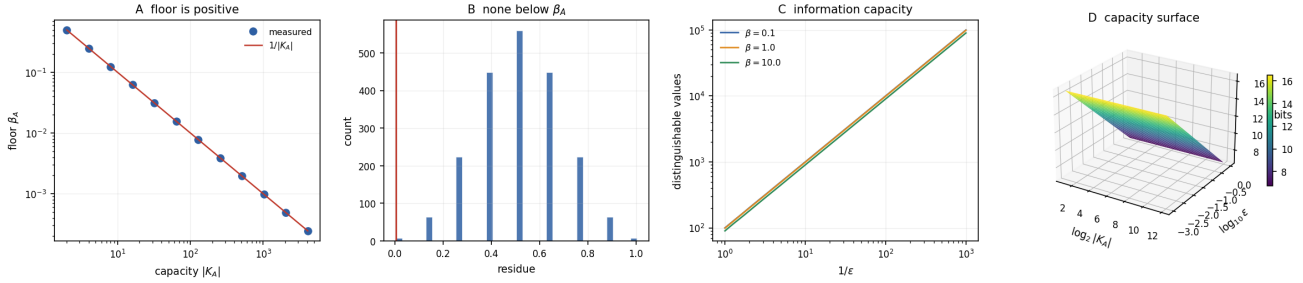

Figure 1: **The residue floor and its information capacity.** The diagonal floor β_A is strictly positive and tracks $1/|K_A|$; no candidate is read below it; and the number of distinguishable residue values on $[\beta_A, U]$ obeys the predicted $\lceil (U - \beta_A)/\varepsilon \rceil$ bound (Theorems 3.1 and 3.3).

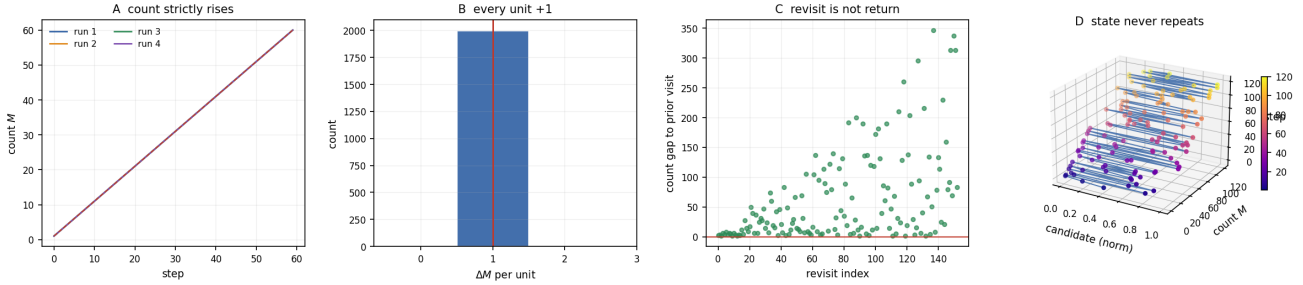

Figure 2: **Strict monotonicity of the trajectory count.** The count increments by exactly one per work unit and never decrements, even when a fraction of units are "undo" operations restoring a prior candidate; revisited candidates yield distinct (x, M) states (Theorem 4.2, Theorem 4.3).

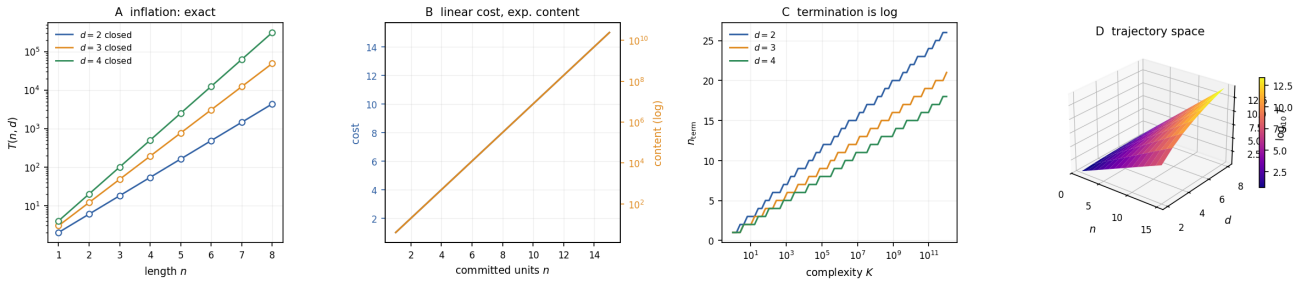

Figure 3: **Trajectory inflation and logarithmic termination.** The closed form $T(n, d) = d(d+1)^{n-1}$ matches direct enumeration exactly; the content grows exponentially in committed units while cost grows linearly; and $n_{\text{term}}(K, d)$ is logarithmic in K (Theorems 5.3 and 5.6).

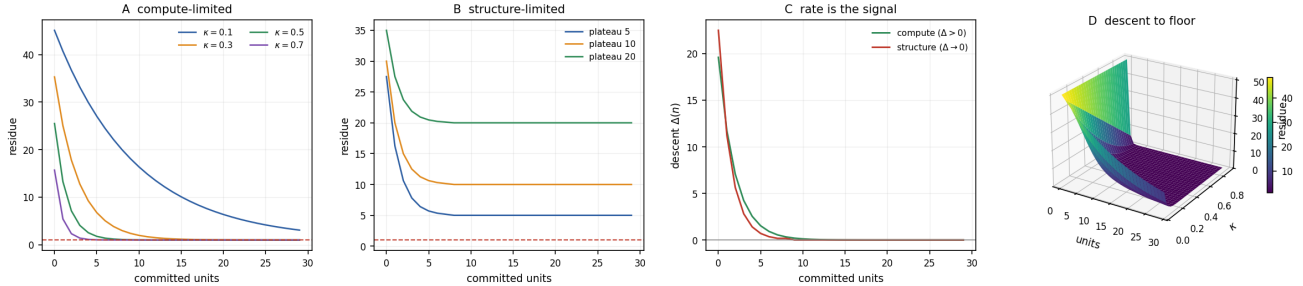

Figure 4: **Runtime diagnosis: compute- vs. structure-limited.** A geometrically descending residue stream ($\Delta > 0$) is read as compute-limited; a stream that plateaus above the floor ($\Delta = 0$) as structure-limited; the classification uses the residue stream alone (Theorem 6.3).

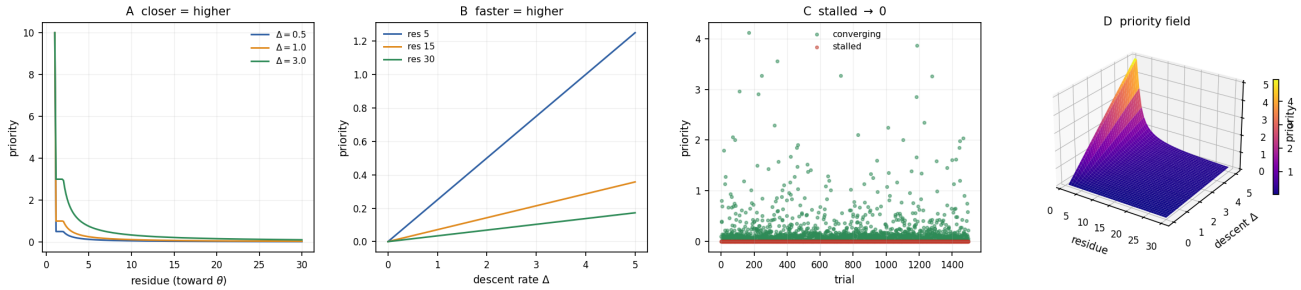

Figure 5: **Residue priority and scheduler soundness.** Priority rises with descent rate and with closeness to threshold, is zero for stalled tasks and unbounded at threshold; over random task sets a stalled task never out-prioritises a converging one (Theorem 7.2, Theorem 7.4).

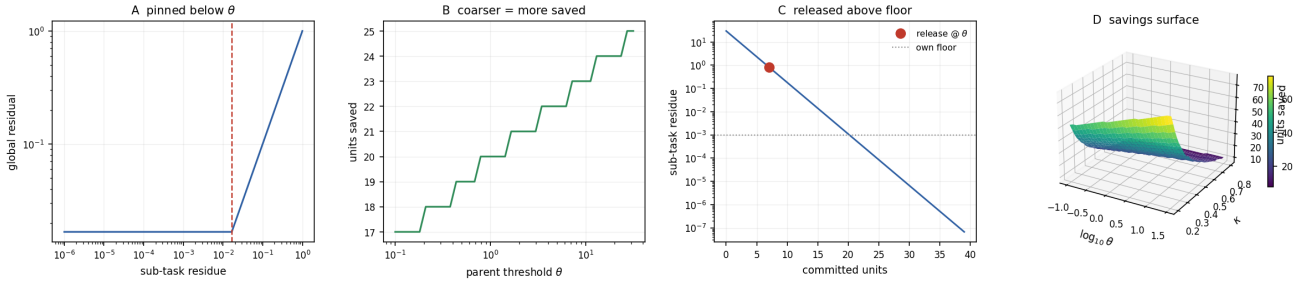

Figure 6: **Sufficiency stopping.** A sub-task released at the parent threshold θ—above its own floor—leaves the global attainable residue unchanged while saving the committed cost of the additional units; the saved cost grows as the parent floor coarsens relative to the sub-task floor (Theorem 8.2).

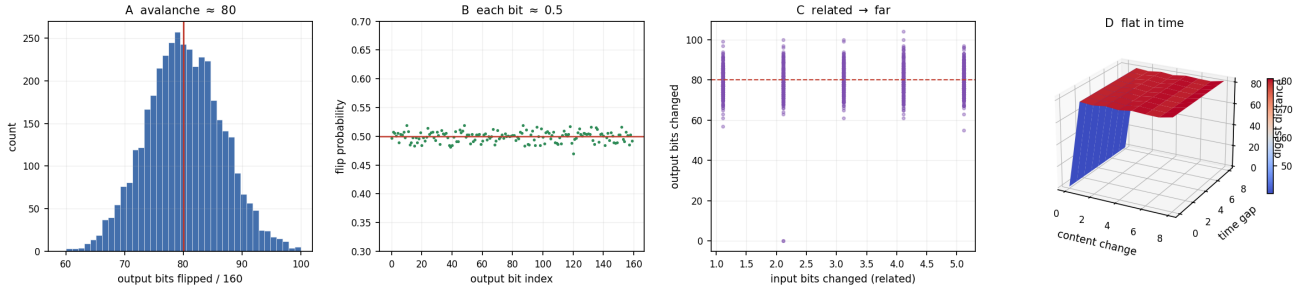

Figure 7: **Avalanche and time-blindness of a content digest.** A single input-bit flip induces a near-half (fraction ≈ 0.5) output bit difference, so related inputs receive maximally distant digests; and identical content at distinct times yields identical digests (Theorems 9.2 and 9.5).

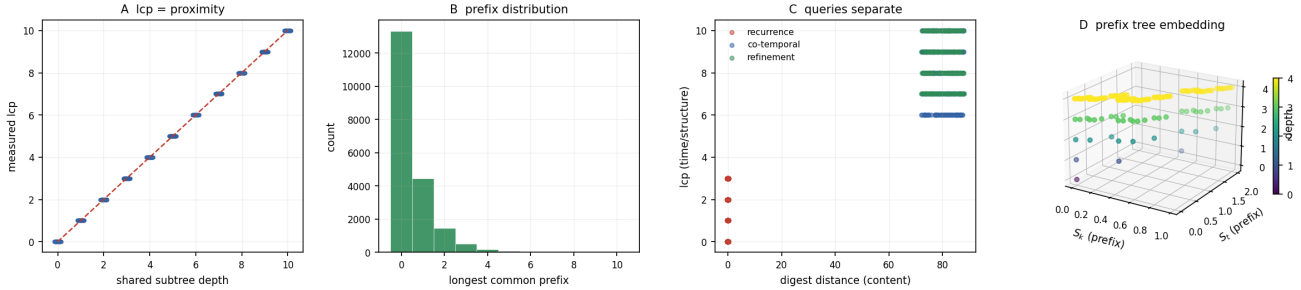

Figure 8: **Prefix is proximity; the paired label.** The longest-common-prefix of two trajectory addresses equals their shared subtree depth; addresses extend monotonically without collision; and the paired content–time label separates recurrence, co-temporality, and refinement (Theorems 9.8 and 9.11).

The unknowable-duration premise is the scheduling face of the absence of a general running-time oracle (Blum, 1967; Turing, 1936).

Hashing and locality. The label orthogonality theorem (Theorem 9.2) is a separation between cryptographic hashing (avalanche (Feistel, 1973; Shannon, 1949; Webster and Tavares, 1986), optimal for identity (Katz and Lindell, 2014; National Institute of Standards and Technology, 2015; Stevens et al., 2017)) and locality-sensitive hashing (proximity-preserving (Broder, 1997; Charikar, 2002; Indyk and Motwani, 1998), optimal for relatedness). We do not propose a hash that does both—we prove none can—and instead pair a content digest with a monotone prefix-structured trajectory address, the latter a structurally enriched logical clock (Fidge, 1988; Lamport, 1978; Mattern, 1989). Time-blindness of a content hash (Theorem 9.5) is, to our knowledge, not usually stated as a theorem though it is implicit in the definition of a content-addressed store (Merkle, 1980, 1988).

Bounded agents. The Residue Floor Theorem (Theorem 3.1) is a finite-capacity result in the spirit of bounded rationality (Simon, 1955): a finite representation cannot resolve a solution to a point, and the irreducible gap is the margin of recognition. The information bound (Theorem 3.3) makes this a sizing law on the residue scale (Cover and Thomas, 2006; Shannon, 1948).

13 Conclusion

We have built a scheduler that does not predict running times—which we argued, and the standard theory confirms, cannot be done in general—and instead orders work by a measured residue and its rate of descent. The construction rests on three proved facts: a bounded agent cannot drive residue to zero, so "solved" is a region of positive thickness $\beta > 0$ whose boundary is the margin of recognition (Theorem 3.1); the count of committed work is strictly monotone, so a process's progress is a faithful non-decreasing coordinate and a revisited state is never a return (Theorem 4.2); and the number of distinguishable work-trajectories grows as $d(d+1)^{n-1}$, giving content-bounded tasks a logarithmic termination bound and a static complexity baseline (Theorems 5.3 and 5.6). The live deviation of residue from this baseline diagnoses, without any duration estimate, whether a task is compute-limited (feed it) or structure-limited (starve it) (Theorem 6.3), and a sub-task is released as soon as it is sufficient for the parent goal, even above its own floor, by a judgment the sub-task itself cannot make (Theorems 8.2 and 8.4). The scheduler's priority rule is sound, livelock-free, and of $\mathcal{O}(\log L)$ overhead per unit (Theorems 7.4, 7.5 and 10.3).

For annotation we proved that identity and relatedness demand opposite locality and so cannot share a label (Theorem 9.2): a cryptographic digest is optimal for identity and destroys relatedness, and is moreover structurally time-blind (Theorem 9.5). Pairing the digest with a monotone, prefix-structured trajectory address supplies the missing temporal/structural axis and makes recurrence, co-temporality, and refinement decidable in $\mathcal{O}(1)$ (Theorem 9.11), with the pairing proved necessary (Theorem 9.12). A process is content-at-a-time; its label must carry both faces, and no single hash can.

The throughline is one quantity: the residue—the positive-thickness gap between a partial result and a recognised solution. It is at once the scheduler's progress signal, the floor that makes recognition possible, the diagnostic that separates compute-limits from structure-limits, and the basis on which a process is annotated by what it is and when it ran. A scheduler that measures the gap, rather than predicting the time, needs neither an oracle it cannot have nor a label that cannot exist.

References

Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

Manuel Blum. A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2):322–336, 1967.

Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of Sequences*, pages 21–29, 1997.

Peter Brucker. *Scheduling Algorithms*. Springer, 5 edition, 2007.

Moses S. Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*, pages 380–388, 2002.

Edward G. Coffman and Leonard Kleinrock. Computer scheduling methods and their countermeasures. *Proceedings of the AFIPS Spring Joint Computer Conference*, 32:11–21, 1968.

Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2 edition, 2006.

Ivan Bjerre Damgård. A design principle for hash functions. In *Advances in Cryptology — CRYPTO '89, LNCS*, volume 435, pages 416–427, 1990.

Thomas Dean and Mark Boddy. An analysis of time-dependent planning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 49–54, 1988.

Horst Feistel. Cryptography and computer privacy. *Scientific American*, 228(5):15–23, 1973.

Colin J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th Australian Computer Science Conference*, pages 56–66, 1988.

Juris Hartmanis and Richard E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, 1965.

Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, pages 604–613, 1998.

Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. Chapman and Hall/CRC, 2 edition, 2014.

Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

C. L. Liu and James W. Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61, 1973.

Friedemann Mattern. Virtual time and global states of distributed systems. In *Parallel and Distributed Algorithms*, pages 215–226. North-Holland, 1989.

Ralph C. Merkle. Protocols for public key cryptosystems. In *Proceedings of the IEEE Symposium on Security and Privacy*, pages 122–134, 1980.

Ralph C. Merkle. A digital signature based on a conventional encryption function. *Advances in Cryptology — CRYPTO '87, LNCS*, 293:369–378, 1988.

National Institute of Standards and Technology. Secure hash standard (shs). FIPS PUB 180-4, 2015.

Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5 edition, 2016.

Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27: 379–423, 623–656, 1948.

Claude E. Shannon. Communication theory of secrecy systems. *The Bell System Technical Journal*, 28(4): 656–715, 1949.

Herbert A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1): 99–118, 1955.

Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3 edition, 2013.

Richard P. Stanley. *Enumerative Combinatorics, Volume 1*. Cambridge University Press, 2 edition, 2012.

Marc Stevens, Elie Bursztein, Pierre Karpman, Ange Albertini, and Yarik Markov. The first collision for full SHA-1. In *Advances in Cryptology — CRYPTO 2017*, volume 10401 of *LNCS*, pages 570–596. Springer, 2017.

Alan M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936.

A. F. Webster and Stafford E. Tavares. On the design of S-boxes. In *Advances in Cryptology — CRYPTO '85, LNCS*, volume 218, pages 523–534, 1986.

Shlomo Zilberstein. Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3):73–83, 1996.